UNIVERSIDAD AUTÓNOMA DE CHIHUAHUA

PROYECTO 1. MODELADO DE LA BASE DE DATOS AUTOS EN MONGODB

ALUMNOS:

- Ana Ruvi Morales Chavez (376899)
- Mauricio Adrian Ochoa Carrillo (377194)

PROGRAMA: INGENIERO EN SISTEMAS COMPUTACIONALES EN HARDWARE

MATERIA: BASE DE DATOS II

PROFESOR: DE LIRA MIRAMONTES JOSE SAUL

FECHA DE ENTREGA: 04/11/2026

Facultad de Ingenieria
Circuito Número I s/n, Nuevo Campus Universitario, Nte. 2, 31125 Chihuahua, Chih. México
Tel. +52 (614) 442 9500

Universidad Autónoma de Chihuahua
C. Escorza 900, Col. Centro 31000
Tel. +52 (614) 439 1500
Chihuahua, Chih. México

1. Objetivo

Modelar en MongoDB la base de datos relacional BDAutos (6 tablas en Oracle) usando estructuras de embedding (documentos incrustados) y linking (referencias), generar el script que crea la base, sus colecciones y los datos de prueba, y conectarla con Python mediante PyMongo. En este documento se explica el análisis del esquema original, por qué decidimos diseñarlo así y el diccionario de datos del modelo nuevo.

2. Análisis del modelo relacional original

2.1 Diagrama entidad-relación

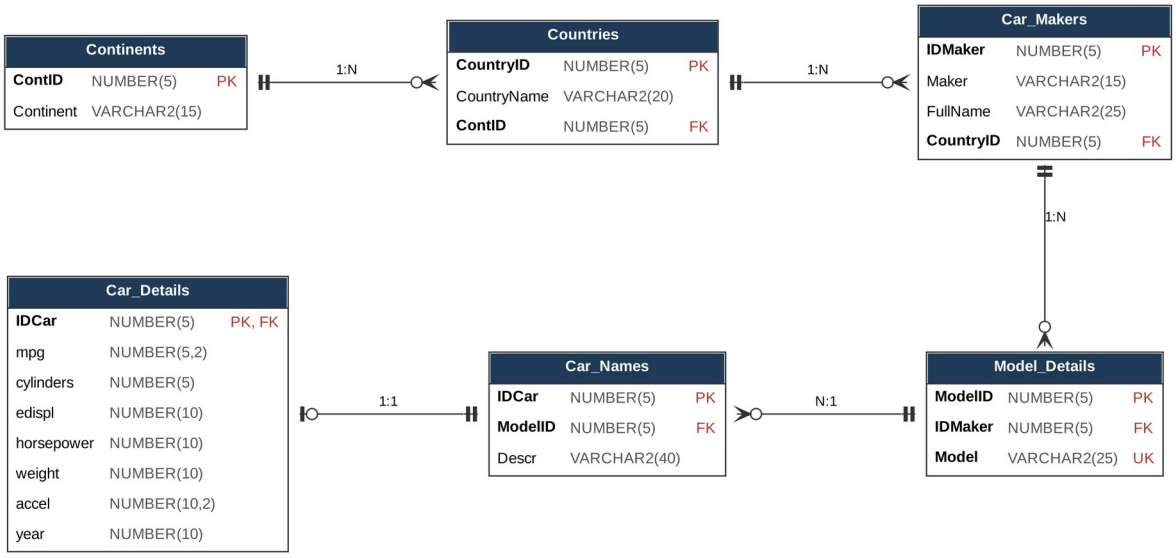


Figura 1. Modelo relacional original de BDAutos (Oracle).

El modelo forma una sola cadena jerárquica: Continents → Countries → Car_Makers → Model_Details → Car_Names → Car_Details. Las primeras cinco relaciones son 1:N y la última es 1:1 (Car_Details comparte la llave IDCar con Car_Names).

2.2 ¿Por qué se diseñó así?

- **Normalización (3FN).** Cada dato se guarda una sola vez: el nombre de un país o continente aparece en un solo renglón y los demás lo referencian por ID. Así no hay redundancia y un cambio se hace en un solo lugar.
- **Catálogos separados.** Continentes, países, fabricantes y marcas son catálogos independientes, lo que facilita darlos de alta aunque todavía no tengan autos (por ejemplo, lamborghini, bentley y dacia no tienen autos registrados).
- **Separación de Car_Names y Car_Details.** Se separa la parte comercial del auto (su descripción) de su ficha técnica. Es una partición vertical típica cuando los datos vienen de fuentes distintas o cuando la ficha técnica se consulta por separado.
- **Integridad referencial.** Las llaves foráneas garantizan que no exista un auto con una marca inexistente, ni una marca con un fabricante inexistente.

2.3 ¿Qué lo hace poco eficiente?

- **Demasiados JOIN.** Para saber de qué continente es un auto hay que unir las 6 tablas (5 JOIN). La mayoría de las 25 consultas pide país o continente, así que casi todas pagarían ese costo.
- **Relación 1:1 innecesaria.** Car_Names y Car_Details siempre se consultan juntas y tienen exactamente los mismos 138 registros; separarlas solo agrega un JOIN más.
- **Catálogos muy pequeños.** Continents tiene 5 registros y Countries 17. Son datos que casi nunca cambian y que siempre se leen junto con el fabricante, así que tenerlos en tablas aparte no aporta mucho.
- **Nombres confusos.** Model_Details no guarda modelos sino marcas (chevrolet, lexus, kia…); el modelo real (silverado, rav4…) está escrito dentro de Descr.
- **Información escondida en texto.** Si el motor es turbo solo se sabe por la "t" al final de Descr, y la cilindrada aparece dos veces: en litros dentro de Descr y en pulgadas cúbicas en edispl.
- **Tipos y nulos poco cuidados.** Las llaves foráneas aceptan nulos, year y weight se declaran como NUMBER(10) y accel como NUMBER(10,2), mucho más grandes de lo necesario.

3. Diseño en MongoDB

La regla que seguimos fue: incrustar lo que siempre se lee junto y casi no cambia, y referenciar lo que tiene vida propia o se consulta por separado. Optimizamos un poco el esquema, pero sin cambiar los datos: se conservan los mismos IDs, nombres, valores y unidades del script original. Resultado: de 6 tablas a 2 colecciones.

3.1 Decisiones de diseño

Tabla original	Destino en MongoDB	Estructura	Razón
`Continents`	makers.country.continent	Embedding	Solo 5 valores fijos; siempre se leen con el país.
`Countries`	makers.country	Embedding	Relación 1:1 desde el fabricante y 17 registros que no cambian.
`Car_Makers`	colección makers	Colección	Entidad principal del catálogo; tiene vida propia.
`Model_Details`	makers.models[ ]	Embedding (arreglo)	Pocas marcas por fabricante (máx. 7); se consultan desde el fabricante.
`Car_Names`	colección cars	Colección	Es lo que más se consulta; 138 documentos.
`Car_Details`	cars.specs	Embedding	Relación 1:1 con Car_Names; siempre se usan juntas.
`FK IDMaker / ModelID`	cars.maker._id / cars.model._id	Linking	Referencia al catálogo para usar $lookup (p. ej., fullName o modelos sin autos).

Referencia extendida. Además del _id del fabricante y de la marca, cada auto guarda una copia de los campos que más se filtran: maker.name, maker.country, maker.continent y

model.name. Con eso, las consultas por país o continente se resuelven en una sola colección, sin $lookup. Esta copia es segura porque son datos que prácticamente no cambian; si un fabricante cambiara de nombre se actualizaría con un solo updateMany. Los datos que no se copiaron, como fullName, se obtienen con $lookup a makers.

¿Por qué no incrustar los autos dentro de makers? Porque el arreglo crecería sin límite y casi todas las consultas trabajan con autos individuales (filtrar, ordenar y agrupar por mpg, hp o cilindros). Tener cars como colección propia hace esas consultas más simples y permite indexarla.

3.2 Diagrama del modelo

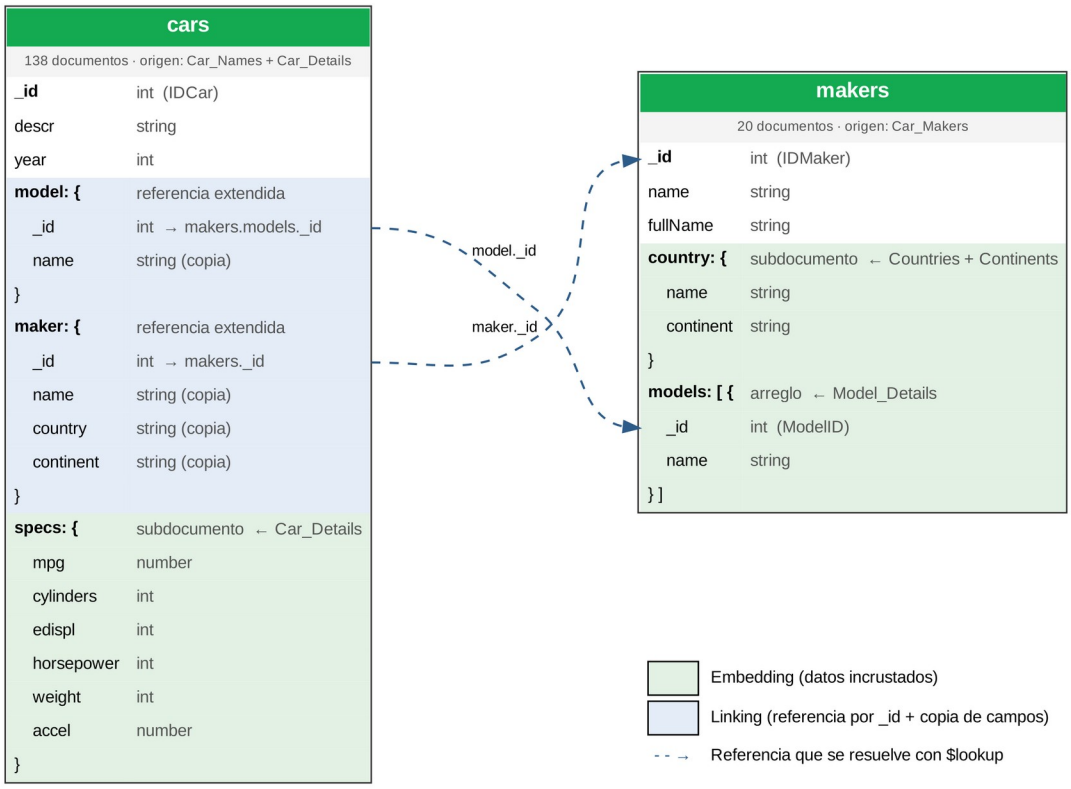


Figura 2. Modelo de BDAutos en MongoDB: verde = incrustado, azul = referencia.

3.3 Ejemplo de documentos

makers (un fabricante con su país y sus marcas incrustadas):

```
{
  _id: 5, name: 'honda', fullName: 'Honda Motor Co.',
  country: { name: 'japan', continent: 'asia' },
  models: [ { _id: 16, name: 'honda' }, { _id: 17, name: 'acura' } ]
}
```

cars (un auto con su ficha técnica incrustada y referencias a su fabricante y marca):

```
{
  _id: 58, descr: 'honda civic ex 1.5t', year: 2024,
  model: { _id: 16, name: 'honda' },
  maker: { _id: 5, name: 'honda', country: 'japan', continent: 'asia' },
  specs: { mpg: 36, cylinders: 4, edispl: 91, horsepower: 180,
           weight: 3000, accel: 7.4 }
}
```

4. Diccionario de datos (MongoDB)

Base de datos: **BDAutos**. Ambas colecciones tienen validación con $jsonSchema, así que MongoDB rechaza documentos sin los campos obligatorios o con tipos incorrectos.

4.1 Colección makers (20 documentos)

Grupos automotrices con su país sede y las marcas que comercializan.

Campo	Tipo	Req.	Origen	Descripción
`_id`	int	Sí	Car_Makers.IDMaker	Identificador único del fabricante.
`name`	string (≤15)	Sí	Car_Makers.Maker	Nombre corto del grupo (gm, toyota…).
`fullName`	string (≤25)	Sí	Car_Makers.FullName	Nombre completo o razón social.
`country`	object	Sí	Countries	País sede del fabricante (incrustado).
`country.name`	string (≤20)	Sí	Countries.CountryName	Nombre del país en inglés, minúsculas.
`country.continent`	string (enum)	Sí	Continents.Continent	america, europe, asia, africa o australia.
`models`	array	Sí	Model_Details	Marcas que comercializa el fabricante (incrustadas).
`models._id`	int	Sí	Model_Details.ModelID	Identificador único de la marca.
`models.name`	string (≤25)	Sí	Model_Details.Model	Nombre de la marca; único en toda la colección.

4.2 Colección cars (138 documentos)

Versiones de autos 2023–2025 con su ficha técnica.

Campo	Tipo	Req.	Origen	Descripción
`_id`	int	Sí	Car_Names.IDCar	Identificador único del auto.
`descr`	string (≤40)	Sí	Car_Names.Descr	Marca, modelo, versión y cilindrada en litros (sufijo t = turbo).
`year`	int	Sí	Car_Details.year	Año modelo.

Campo	Tipo	Req.	Origen	Descripción
`model._id`	int	Sí	Car_Names.ModelID	Referencia a makers.models._id.
`model.name`	string	Sí	Model_Details.Model	Copia del nombre de la marca.
`maker._id`	int	Sí	Model_Details.IDMaker	Referencia a makers._id.
`maker.name`	string	Sí	Car_Makers.Maker	Copia del nombre del fabricante.
`maker.country`	string	Sí	Countries.CountryName	Copia del país del fabricante.
`maker.continent`	string	Sí	Continents.Continent	Copia del continente del fabricante.
`specs.mpg`	int / double	Sí	Car_Details.mpg	Rendimiento combinado en millas por galón.
`specs.cylinders`	int	Sí	Car_Details.cylinders	Número de cilindros.
`specs.edispl`	int	Sí	Car_Details.edispl	Cilindrada en pulgadas cúbicas.
`specs.horsepower`	int	Sí	Car_Details.horsepower	Potencia en caballos de fuerza (hp).
`specs.weight`	int	Sí	Car_Details.weight	Peso en libras.
`specs.accel`	int / double	Sí	Car_Details.accel	Segundos de 0 a 60 mph.

4.3 Índices

Índice	Colección	Para qué sirve
`{ _id: 1 }`	ambas	Automático; equivale a la PK original.
`{ 'maker._id': 1 }`	cars	$lookup desde makers (autos por fabricante).
`{ 'model._id': 1 }`	cars	$lookup desde makers.models (autos por marca).
`{ 'maker.continent': 1, 'maker.country': 1 }`	cars	Filtros y agrupaciones por continente y país.
`{ 'models._id': 1 } único`	makers	Que no se repita el ID de una marca.
`{ 'models.name': 1 } único`	makers	Equivale a UNIQUE(Model) del original.

5. Script de la base de datos

El archivo **01_crear_bdautos.js** se corre con `mongosh 01_crear_bdautos.js` (o se pega en la shell de MongoDB Compass). Hace lo siguiente, en orden:

- Crea la base BDAutos (si ya existe, la borra para empezar limpio).
- Crea las colecciones makers y cars con su validación $jsonSchema.
- Inserta los 20 fabricantes (con sus 42 marcas) y los 138 autos con insertMany.
- Crea los índices de la sección 4.3.
- Imprime una verificación de conteos.

Al correrlo obtuvimos los mismos totales que el script original:

```
makers: 20 (esperado 20)
modelos: 42 (esperado 42)
cars: 138 (esperado 138)
```

También comparamos varios resultados contra la base relacional original (promedios por país, autos por categoría, autos turbo japoneses) y dieron exactamente lo mismo, así que la migración no perdió ni cambió datos.

6. Conexión de Python con MongoDB (PyMongo)

Se instala el driver con `pip install pymongo`. El archivo **conexion.py** tiene la función que abre la conexión y comprueba que el servidor responda:

```
from pymongo import MongoClient

def get_db(uri, db_name):
    client = MongoClient(uri, serverSelectionTimeoutMS=5000)
    client.admin.command("ping")
    return client[db_name]

db = get_db("mongodb://localhost:27017", "BDAutos")
```

Con la base en la variable db, las consultas se escriben con diccionarios de Python, igual que en mongosh pero con las claves entre comillas:

```
# find con filtro, proyección y orden
resultado = list(db.cars.find(
    {"year": {"$gte": 2024}, "specs.mpg": {"$gte": 35}},
    {"_id": 0, "descr": 1, "year": 1, "specs.mpg": 1}
).sort("specs.mpg", -1))

# aggregate con un pipeline
pipeline = [
    {"$group": {"_id": "$maker.country", "autos": {"$sum": 1}}},
    {"$sort": {"autos": -1}}
]
resultado = list(db.cars.aggregate(pipeline))
```

La base también se puede crear desde Python con **crear_bd.py** (create_collection, insert_many y create_index). Las 25 consultas están en **consultas.py** y se muestran y ejecutan en la aplicación de escritorio.